Podrobna razlaga: Sistem zajemanja slik z RedPitayo

Razširjena dokumentacija za začetnike

Pripravljeno na podlagi izvirne dokumentacije Martina Horvata

17. september 2025 (razširjena razlaga)

Kazalo

Uvod – kaj sploh počne ta sistem? **2**

1 FPGA **4**

Kaj je FPGA in zakaj Verilog? . . . 4
Zgradba sistema – pregled blokov . . . 4
Kako brati blokovno shemo (system.pdf) . . . 4
1.1 Kontrolni parametri . . . 5
AXI protokol – "jezik", s katerim komunicirajo bloki . . . 5
Kaj je "register" v tem kontekstu? . . . 5
Xilinxov blok reg_ctrl (Slika 4) . . . 5
Branje parametrov (Slika 1) . . . 6
Zapisovanje parametrov (Slika 2) . . . 6
Zakaj potrebujemo ločeno branje in pisanje? . . . 7
Definicija lokacij parametrov (Slika 3) . . . 7
Kaj je "GP port" fizično? . . . 8
1.2 Zajem signala . . . 8
ADC – analogno-digitalni pretvornik . . . 8
Integrator – zakaj sešteva signal? . . . 8
Blok acquire_top – "dirigent" zajema . . . 9
Koda za proženje zapisa in ponastavitev integratorja (Slika 5) . . . 9
1.3 Generiranje signala . . . 10
Skupni pomnilnik za nastavitve in vzorce . . . 10
Blok waveform_top in dva "generatorja spomina" . . . 10
Kako se vrednost zapiše na pravo mesto (Slika 7) . . . 11
Od pomnilnika do DAC izhoda . . . 11
1.4 Komunikacija zajetega signala . . . 12
Zakaj GP port ni dovolj hiter? . . . 12
Zgradba bloka AXI_HP_top . . . 12
Kaj je FIFO in zakaj ga potrebujemo? . . . 12
Sestavljanje FIFO zapisa (Slika 8) . . . 13
Zapis v FIFO (Slika 9) . . . 13
Zastavice za iniciacijo prenosa (Slika 10) . . . 14
Interrupt – kako FPGA "pokliče" procesor . . . 14

2 DeviceTree **15**
Zakaj Linux potrebuje “DeviceTree”? 15
Kako se naslovi ujemajo med Vivadom in DeviceTree (Slika 11) 15
DeviceTree overlay (Slika 12) 15
Kompilacija in nalaganje overlay-a 17

3 C program **18**
Kaj je UIO in zakaj ga uporabljamo? 18
3.1 Identifikacija UIO naprav 18
Kako Linux poimenuje UIO naprave 18
Več “map” (`maps`) znotraj ene UIO naprave 19
3.2 Mapiranje registrov in bufferja 19
Funkcija za dodajanje generatorja (Slika 13) 19
Zakaj deluje `mmap()` tukaj tako “magično”? 20
Definicija struktur (Slika 14) 20
3.3 Prenos FIFO v RAM 21
Poslušanje za interrupt (Slika 15) 21
Zakaj je pomembno, da se to dogaja v neskončni zanki? 22
Brisanje interrupta in zaprošanje za novega (Slika 16) 22
3.4 TCP komunikacija 23
Kaj je TCP in zakaj se uporablja tukaj? 23
Celoten potek podatkov, od FPGA do slike 23

Povzetek najpomembnejših pojmov **25**

Uvod – kaj sploh počne ta sistem?

Preden se poglobimo v posamezna poglavja, si na kratko oglejmo celotno sliko. RedPitaya je majhna razvojna ploščica, ki v sebi združuje dve zelo različni stvari:

1. **FPGA (Field-Programmable Gate Array)** – čip, ki ga lahko “sprogramiramo” tako, da si sam sestavi svojo notranjo vezje. FPGA ne izvaja programa korak za korakom kot procesor, ampak je v resnici ogromno mrežo logičnih vrat in pomnilnikov, ki so povezana tako, kot smo jih mi opisali v kodi (Verilog/SystemVerilog). Vse, kar je opisano v FPGA delu, se dogaja **hkrati**, vsak takt ure (angl. *clock cycle*).
2. **Procesorski sistem (PS)** – to je “navaden” ARM procesor, ki poganja Linux. Na njem teče C program, ki komunicira s FPGA delom.

Namen celotnega sistema je: zajeti signal iz nekega detektorja (npr. svetlobni detektor), hkrati zajeti tudi signal položaja “hitrega ogledala” (mirror), ta dva signala povezati skupaj v en zapis, prenesti ta zapis v pomnilnik (RAM), od tam ga prebrati s procesorjem in ga preko omrežja (TCP) poslati na osebni računalnik, kjer se iz teh podatkov sestavi slika. Poleg tega sistem znotraj FPGA tudi generira signale (npr. za krmiljenje ogledala) preko DAC izhodov.

Slikovito povedano: gre za mini-oscilograf in mini-generator signalov v enem, ki ju upravljamo preko Linuxa in ju povežemo z zunanjim svetom preko omrežja.

V nadaljevanju bomo šli skozi popolnoma enaka poglavja kot v izvirni dokumentaciji, le da bo vsako poglavje razloženo veliko bolj podrobno – tako, kot bi razlagali nekomu, ki se s FPGA, Verilogom, Linuxom in vgrajenimi sistemi šele spoznava.

1 FPGA

Program na FPGA je napisan v jeziku **Verilog/SystemVerilog**, z orodjem **Vivado 2020.2** (to je Xilinxovo razvojno okolje za načrtovanje FPGA vezij). Preden gremo v podrobnosti, razložimo osnovne pojme.

Kaj je FPGA in zakaj Verilog?

FPGA si lahko predstavljamo kot ogromno škatlo z tisoči praznih logičnih gradnikov (AND, OR, NOT vrata, flip-flopi, majhni koščki pomnilnika…) in programirljivimi povezavami med njimi. Ko "programiramo" FPGA, v resnici ne pišemo zaporedja ukazov (kot pri C programu), ampak **opisujemo strojno vezje** – kako naj bodo ti gradniki med seboj povezani. Ko je FPGA enkrat konfiguriran, vezje deluje vzporedno in neprekinjeno, sinhronizirano z eno ali več urinimi signali (*clock*).

Verilog/SystemVerilog je jezik, s katerim to vezje opišemo. Namesto "naredi to, nato to", pišemo stavke tipa "vsak takt ure naj se zgodi to" (`always @(posedge clk)`), kar dejansko pomeni "ob vsakem naraščajočem robu urinega signala naj se posodobijo te vrednosti".

Zgradba sistema – pregled blokov

Sistem je sestavljen iz več **modulov** (v Verilogu jim pravimo tudi "top" moduli, ker so vsak "vrh" svojega dela hierarhije). Vsak modul opravlja svojo nalogo, med seboj pa so povezani prek žic (signalov). Bloki so:

- **`reg_ctrl_top`** – "nadzorna plošča" celotnega sistema. Skrbi za to, da lahko procesor (Linux program) prebere ali spremeni nastavitve (parametre) FPGA vezja.
- **`acquire_top`** in **`integrator_top`** – skrbita za zajem signalov iz analogno-digitalnega pretvornika (ADC) in njihovo obdelavo (integracijo).
- **`generate_top`** in **`waveform_top`** – skrbita za generiranje poljubnih signalov, ki gredo nato na digitalno-analogni pretvornik (DAC).
- **`AXI_HP_top`** – skrbi za hiter prenos velikih količin zajetih podatkov v RAM pomnilnik preko
  t. i. High Performance vodila.

Vse te module (in še nekaj pomožnih, kot je `input_selector`, `integrator_top`, `vector_split`, `generator_test`) lahko vidimo tudi v blokovni shemi, ki je shranjena v datoteki `system.pdf` – to je slika, ki grafično prikaže, kako so vsi ti "kvadratki" med seboj povezani z žicami (puščicami). Ta shema je v bistvu enak sistem, kot ga opisuje koda, le da je narisan vizualno, kar zelo pomaga pri razumevanju povezav.

Kako brati blokovno shemo (system.pdf)

Na sredini shranjenega diagrama je največji blok **`processing_system7`** – to je sam ARM procesor (Zynq). Vse ostalo okoli njega so bloki, ki jih smo mi napisali (t. i. "custom IP") ali pa so Xilinxovi vgrajeni bloki (npr. `axi_interconnect`, `clk_wiz` – generator urinega signala, ali `proc_sys_reset` – skrbi za resetiranje sistema). Puščice med bloki

predstavljajo signale: nekatere so tanke (posamezni biti, npr. `trig`, `start_acq`), druge so debelejše in imajo oznako v oglatih oklepajih, npr. `[31:0]`, kar pomeni, da gre za 32-bitni vodilo (32 žic hkrati, ki skupaj predstavljajo eno število).

1.1 Kontrolni parametri

To poglavje razloži, **kako Linux program "govori" s FPGA vezjem** – torej kako lahko iz C kode nastavimo npr. hitrost zajema signala, ali preberemo, koliko podatkov je že bilo zajetih.

AXI protokol – "jezik", s katerim komunicirajo bloki

AXI (Advanced eXtensible Interface) je standardiziran protokol/vodilo, ki ga uporablja skoraj vsak Xilinx/ARM sistem za komunikacijo med procesorjem in strojno opremo. Predstavljajte si ga kot poštni sistem: procesor napiše "pismo" (naslov + podatek), AXI vodilo poskrbi, da pride pismo na pravo mesto, in vrne "potrdilo o prejemu".

Obstajata (med drugim) dve različici tega vodila, ki ju ta sistem uporablja:

- **GP (General Purpose)** – "počasnejši", a enostavnejši način komunikacije, primeren za nastavljanje parametrov (npr. "nastavi decimacijo na 100").
- **HP (High Performance)** – hitrejši način, namenjen prenosu velikih količin podatkov (npr. celotnega zajetega signala) direktno v RAM. O tem govori poglavje 1.4.

V tem podpoglavju gre za GP komunikacijo, torej za "počasne", a preproste ukaze tipa "preberi/napiši register".

Kaj je "register" v tem kontekstu?

Register je preprosto majhno pomnilniško mesto (npr. 32 bitov = eno celo število) znotraj FPGA, ki ima svoj **naslov** (address). Procesor lahko na ta naslov napiše vrednost (s tem "pove" FPGA-ju, kaj naj naredi ali kako naj se nastavi), ali pa z tega naslova prebere vrednost (s tem "vpraša" FPGA, kakšno je trenutno stanje). To je popolnoma analogno temu, kako procesor komunicira z navadnim RAM-om, le da so tu "pomnilniške celice" v resnici žice, ki neposredno vplivajo na delovanje logike v FPGA.

Xilinxov blok reg_ctrl (Slika 4)

Blok `reg_ctrl` je Xilinxov vgrajen (že narejen) gradnik, ki poskrbi za vso "administracijo" AXI protokola – torej za rokovanje s signali, kot so `s_axi_awvalid`, `s_axi_wdata`, `s_axi_bresp` ipd. Vsi ti signali s predpono `s_axi_` so del standarda AXI in skrbijo za:

- **naslavljanje** (na kateri naslov pišemo/beremo – `awaddr`, `araddr`),
- **prenos podatkov** (`wdata` za pisanje, `rdata` za branje),
- **usklajevanje časovanja** (signali `valid`/`ready` – pošiljatelj pove "podatek je pripravljen", prejemnik pa "jaz sem pripravljen sprejeti").

Namesto da bi mi sami pisali vso to logiko usklajevanja (kar je precej zamudno in dovzetno za napake), uporabimo Xilinxov že preverjen blok `reg_ctrl`, ki znotraj sebe to pretvori v veliko preprostejši vmesnik: navaden **BRAM** (Block RAM) vmesnik s signali `bram_addr_a`, `bram_wrdata_a`, `bram_rddata_a`, `bram_we_a` (write enable – "ali naj se zapiše") itd. To je veliko lažje razumeti in uporabljati v naši lastni logiki.

Branje parametrov (Slika 1)

Zdaj si oglejmo kodo za branje:

```
always @(posedge clk)
begin
  case (reg_addr[16-1:0])
    DEC_FACTOR_ADDR:        reg_rd_data <= cfg_dec;
    START_ACQ_ADDR:         reg_rd_data <= {31'b0, start_acq};
    DEST_ADDR:              reg_rd_data <= dest_addr;
    ...
    default:                reg_rd_data <= wfm_read;
  endcase
end
```

Razložimo vsak del:

- `always @(posedge clk)` pomeni "ob vsakem naraščajočem robu urine signala `clk` se izvede spodnji blok". To ni "funkcija", ki se pokliče – to je opis vezja, ki se **vedno** izvaja, vsak takt.
- `case (reg_addr[16-1:0])` je kot `switch` v C-ju: glede na vrednost naslova `reg_addr` (prvih 16 bitov) izberemo, kaj naj se zgodi.
- Vsaka vrstica, npr. `DEC_FACTOR_ADDR: reg_rd_data <= cfg_dec;`, pomeni: "če je naslov enak vrednosti `DEC_FACTOR_ADDR`, potem naj bo vrednost, ki jo bomo vrnili procesorju (`reg_rd_data`), enaka trenutni vrednosti notranje spremenljivke `cfg_dec`."
- Operator `<=` v Verilogu ni "manjše ali enako", ampak **neblokirajoča dodelitev** – pomeni "ob naslednjem urinem taktu naj ta signal dobi to vrednost". To je ena izmed ključnih razlik med Verilogom in navadnimi programskimi jeziki: dodelitve se ne zgodijo "takoj", ampak sinhronizirano z uro.
- `default: reg_rd_data <= wfm_read;` pomeni "če naslov ni nobeden od zgoraj naštetih, privzeto vrni vrednost `wfm_read`" – to se uporabi za branje vzorcev funkcije, ki jo generiramo (glej poglavje 1.3).

Torej: ko procesor "prebere" določen naslov preko AXI, ta koda poskrbi, da se na izhodu `reg_rd_data` pojavi prava vrednost, ki jo nato `reg_ctrl` blok "spakira" nazaj v AXI odgovor in posreduje procesorju.

Zapisovanje parametrov (Slika 2)

```
always @(posedge clk)
begin
  if (rst_n == 0) begin
```

```
    cfg_dec <= 1;
  end else begin
    if ((reg_addr[16-1:0] == DEC_FACTOR_ADDR) && (reg_wr_we == 1)) begin
      cfg_dec <= reg_wr_data;
    end
  end
end
```

Tu gre za nasprotni postopek – **pisanje**:

- `if (rst_n == 0)` – `rst_n` je signal za resetiranje sistema (ime s pripono `_n` po dogovoru pomeni "aktiven, ko je 0" – *active low*). Če je sistem v resetu, se `cfg_dec` (faktor decimacije) postavi na privzeto vrednost 1.
- Sicer (`else`), če je naslov, ki ga procesor pošilja, enak `DEC_FACTOR_ADDR` **in** je hkrati postavljen signal `reg_wr_we` (*write enable* – "zapisovanje je omogočeno/aktivno"), potem se notranja spremenljivka `cfg_dec` posodobi na vrednost, ki jo je procesor poslal (`reg_wr_data`).

Pomembno je razumeti, da imamo v resnici toliko takšnih `if` stavkov, kolikor je zapisljivih parametrov (v resnici bi jih v polni kodi videli enega za vsak naslov) – dokumentacija prikaže le enega kot primer, princip pa se ponovi za vsak parameter (npr. `START_ACQ_ADDR`, `DEST_ADDR`, `BUFF_SIZE_ADDR` itd.).

Zakaj potrebujemo ločeno branje in pisanje?

V strojni opremi branje in pisanje nista "isto vrata" kot v programski opremi (kjer je spremenljivka preprosto spremenljivka). Fizično imamo:

- **register (flip-flop)**, ki hrani vrednost (npr. `cfg_dec`) – to je "prava" spremenljivka, ki jo lahko FPGA logika kadarkoli uporablja (npr. integrator jo uporabi za nastavitev decimacije).
- **logiko za pisanje** – nadzoruje, kdaj in kako se ta register spremeni (na osnovi ukaza s procesorja).
- **logiko za branje** – nadzoruje, kaj se procesorju vrne, ko ta "vpraša" za vrednost na določenem naslovu.

Te tri stvari so ločene, ker morajo delovati vzporedno in neprestano – register mora obstajati zmeraj (ne le takrat, ko ga nekdo prebere), logika branja se mora odzvati takoj, ko pride poizvedba, logika pisanja pa mora "čuvati" register in ga spremeniti le, kadar je to izrecno zahtevano.

Definicija lokacij parametrov (Slika 3)

```
localparam DEC_FACTOR_ADDR        = 16'h0000;
localparam START_ACQ_ADDR         = 16'h0004;
localparam DEST_ADDR              = 16'h0008;
...
localparam START_GEN_ADDR         = 16'h1000;
```

```
...
localparam SHIFT_WFM              = 16'h2000;
```

`localparam` v Verilogu pomeni "konstanta, ki velja znotraj tega modula" – torej vrednost, ki se med delovanjem sistema nikoli ne spremeni (podobno kot `#define` ali `const` v C-ju). Tu so definirani vsi naslovi (v šestnajstiškem, hex, zapisu – oznaka `16'h...` pomeni "16-bitno število v hex obliki"), na katerih se posamezni parametri "nahajajo" znotraj naslovnega prostora, ki ga vidi AXI GP vodilo.

Zelo pomembno je opaziti vzorec: naslovi od `0x0000` do `0x002C` so namenjeni **kontrolnim parametrom** zajema (decimacija, start/stop zajema, cilj v RAM, velikost bufferja...), naslovi od `0x1000` do `0x1010` so namenjeni **parametrom generiranja** signala (dolžine valovnih oblik, decimacije generatorja), naslov `0x2000` (`SHIFT_WFM`) pa je meja, od koder naprej naslovni prostor ni več namenjen "parametrom", ampak dejanskim **vzorcem signala**, ki ga želimo generirati (glej poglavje 1.3). To je pomembno, ker isti del pomnilnika (isti AXI vmesnik) uporabljamo za dva različna namena, ločena le po tem, kateri naslov uporabimo.

Kaj je "GP port" fizično?

`reg_ctrl_top` je povezan na **`M_AXI_GP0`** izhod procesorja (`processing_system7` blok na blokovni shemi) preko `axi_interconnect` bloka. `axi_interconnect` je nekakšno "stikalo" ali "razdelilnik", ki lahko en AXI vhod poveže z enim ali več AXI izhodi (in obratno) – v tem primeru poveže procesorjev GP izhod s našim `reg_ctrl_top` vhodom.

1.2 Zajem signala

To poglavje pojasni, kako sistem dejansko **zajame** analogni signal in ga pripravi za shranjevanje.

ADC – analogno-digitalni pretvornik

RedPitaya ima na vhodu dva analogna kanala, ki gresta skozi **ADC (Analog-to-Digital Converter)**. ADC neprestano (vsak urin cikel) pretvarja napetost na vhodu v digitalno število (na blokovni shemi vidimo signale `adc_data_ch1[13:0]` in `adc_data_ch2[13:0]` – 14-bitna števila, torej lahko ADC loči 2^14 = 16384 različnih nivojev napetosti).

V tem sistemu:

- **En ADC vhod** je povezan na **detektor** (npr. fotodetektor, ki zaznava svetlobo ali kakšen drug fizični signal, ki ga snemamo).
- **Drugi ADC vhod** je povezan na senzor, ki pove **položaj hitrega ogledala** (angl. *fast mirror* – naprava, ki se hitro premika in "skenira" žarek po neki sceni, podobno kot žarek v starem CRT televizorju skenira po zaslonu).

Integrator – zakaj sešteva signal?

Signal iz detektorja gre skozi **integrator** (`integrator_top` blok). Integracija pomeni seštevanje (kopičenje) vrednosti signala skozi čas. Zakaj je to koristno? Ker je

posamezen vzorec ADC lahko šumen (naključne majhne napake), medtem ko vsota (integral) več zaporednih vzorcev bolje predstavlja "resnično" energijo signala v določenem časovnem oknu – podobno kot bi namesto ene same fotografije naredili dolgo eksponiran posnetek, ki zajame več svetlobe in je manj šumen.

Dokumentacija omenja, da integrator sešteva **po trapeznem pravilu** (angl. *trapezoidal rule*) – to je matematična metoda numerične integracije, kjer namesto da bi preprosto seštevali vrednosti vzorcev (t. i. Riemannova vsota), vzamemo povprečje dveh sosednjih vzorcev za vsak časovni interval, kar bolje aproksimira "resnično" ploščino pod krivuljo signala (manj napake pri numerični integraciji).

Blok acquire_top – "dirigent" zajema

`acquire_top` je blok, ki dobi na vhod:

1. integrirani signal iz detektorja (`integrated_signal[31:0]`),
2. dva pozicijska signala hitrega ogledala,
3. vrednost "počasnega" signala (verjetno kakšen dodaten status ali merilni signal, ki se spreminja počasneje).

Njegova naloga je, da **vsakih `dec_cnt` ciklov ure** sproži zapis teh treh vrednosti naenkrat, kot en "paket" podatkov. Beseda "decimacija" (dec – od *decimate*) v digitalni obdelavi signalov pomeni **zmanjšanje hitrosti vzorčenja** – če ADC vzorči, recimo, 125 milijonov vzorcev na sekundo, ampak mi za našo aplikacijo ne potrebujemo tako gostih podatkov, lahko z decimacijo shranimo le vsak n-ti vzorec (kjer je n = `dec_cnt`), s čimer zmanjšamo količino podatkov, ki jih je treba prenesti in shraniti.

Koda za proženje zapisa in ponastavitev integratorja (Slika 5)

```
always @ (posedge clk) begin
    if (~rst_n || ~start_acq)
        dec_cnt <= 0;
    else if (trig)
        dec_cnt <= 1;
    else
        dec_cnt <= dec_cnt + 32'b1;
end
```

Po vrsti:

- `~rst_n || ~start_acq` – znak ~ v Verilogu pomeni logični NOT (negacija). Če sistem ni v resetu (`rst_n` je 0, kar pomeni "reset je aktiven") **ali** zajem ni zagnan (`start_acq` je 0), se števec `dec_cnt` postavi na 0 – torej "ne štej, dokler ni zajem dejansko zagnan".
- `else if (trig)` – ko pride signal proženja (`trig` – to je signal, ki pove "zdaj je čas za nov zapis vzorca"), se števec ponastavi na 1 (začne štetje novega intervala).
- `else` – v vseh drugih primerih se števec preprosto povečuje za 1 vsak takt ure (`dec_cnt <= dec_cnt + 1`).

Signal `trig` se v resnici sproži takrat, ko `dec_cnt` doseže vrednost `cfg_dec` (parameter, ki smo ga nastavili preko `reg_ctrl_top` – glej poglavje 1.1, naslov `DEC_FACTOR_ADDR`). To pomeni: sistem šteje ure ure, in ko doseže nastavljeno mejo (decimacijski faktor), sproži zapis enega vzorca v FIFO (glej poglavje 1.4) in hkrati ponastavi integrator, da lahko začne seštevati naslednji časovni interval znova od nič.

Skratka: `dec_cnt` je "ura" znotraj FPGA, ki šteje takte in vsakih `cfg_dec` taktov "poklestne z zvoncem" (sproži `trig`), kar sproži verigo dogodkov: zapis vzorca in ponastavitev integratorja.

1.3 Generiranje signala

To poglavje razloži, kako sistem **generira poljubno obliko signala**, ki se nato pošlje na izhodni DAC (Digital-to-Analog Converter) – npr. za krmiljenje hitrega ogledala ali kakšnega drugega aktuatorja.

Skupni pomnilnik za nastavitve in vzorce

Zanimiv (in za začetnike morda zmeden) del je, da se **isti del RAM-a**, ki ga uporabljamo za nastavljanje kontrolnih parametrov (poglavje 1.1), uporablja tudi za shranjevanje dejanskih vzorcev funkcije, ki jo želimo generirati. Ločimo ju s pomočjo naslovne meje `SHIFT_WFM` (`0x2000` iz Slike 3):

- Naslovi **pod** `SHIFT_WFM` → kontrolni parametri (glej poglavje 1.1).
- Naslovi **nad** `SHIFT_WFM` → dejanski vzorci arbitrarne (poljubne) funkcije.

Znotraj tega drugega dela je prostor razdeljen še naprej:

- **Prvih 16384 vzorcev** je namenjenih **prvi** izhodni funkciji (npr. DAC kanal A).
- **Naslednjih 16384 vzorcev** je namenjenih **drugi** izhodni funkciji (DAC kanal B).

Število 16384 = 2^14 ni naključno – to je tipična globina (velikost) enega BRAM bloka na Xilinx FPGA čipih, kar je zelo priročna in učinkovita velikost za tak "waveform" pomnilnik.

Blok waveform_top in dva "generatorja spomina"

`waveform_top` vsebuje **dva Xilinxova generatorja spomina** (angl. *Block Memory Generator*, BRAM) – enega za vsak izhodni kanal (Slika 6):

```
blk_mem_gen_0 #() blk_mem_inst_1 (
    .addra(bram_write_addr[14-2:0]),
    .dina(bram_din),
    .douta(bram_read_1),
    .wea(bram_we_1),
    .clka(clk),
    .addrb(bram_read_addr_1),
    .dinb(16'b0),
    .doutb(bram_dout_1),
    .web(1'b0),
```

```
    .clkb(clk)
);
```

Tu vidimo zelo pomemben in pogost vzorec v FPGA načrtovanju: **dvo-vratni pomnilnik** (angl. *dual-port RAM*). Ta ima dva popolnoma neodvisna "para" vrat – vrata A (`addra`, `dina`, `douta`, `wea`) in vrata B (`addrb`, `dinb`, `doutb`, `web`). To pomeni, da lahko **hkrati**, v istem urinem ciklu:

- preko vrat **A** procesor **zapisuje** nove vzorce funkcije (`wea` = write enable, omogoča pisanje), in
- preko vrat **B** logika za generiranje **bere** vzorce, ki jih pošilja na DAC (`web = 1'b0` pomeni, da se preko vrat B nikoli ne piše, torej so vrata B namenjena izključno branju).

To je odlična rešitev, ker se pisanje (počasno, sporadično, ko uporabnik nastavlja novo obliko signala) in branje (hitro, neprestano, vsak takt, za generiranje izhoda) medsebojno sploh ne motita – dogajata se preko fizično ločenih vrat istega pomnilniškega bloka.

Kako se vrednost zapiše na pravo mesto (Slika 7)

```
assign bram_we_1 = ~bram_write_addr[ADDR_WIDTH-1] && bram_we;
assign bram_we_2 = ~bram_we_1 && bram_we;

assign bram_read = bram_write_addr[ADDR_WIDTH-1] ? bram_read_2 : bram_read_1;
```

`assign` v Verilogu pomeni "ta signal je *vedno* enak temu izrazu" – to ni dodelitev, ki bi se zgodila enkrat, ampak neprekinjena, kombinacijska logika (brez ure), ki se takoj (brez zamika) prilagodi vsaki spremembi vhodov.

- Najvišji bit naslova (`bram_write_addr[ADDR_WIDTH-1]`) nam pove, ali pišemo v prvo ali drugo polovico naslovnega prostora (torej v prvi ali drugi izhodni kanal – spomnimo se delitve na 16384 + 16384 vzorcev).
- `bram_we_1` je aktiven, kadar splošni signal za pisanje (`bram_we`) je aktiven **in** je najvišji bit naslova 0 (torej gre za prvi kanal).
- `bram_we_2` je aktiven, kadar je `bram_we` aktiven, a `bram_we_1` **ni** aktiven – torej gre za drugi kanal.
- Vrstica z operatorjem `?:` je t. i. **pogojni izraz** (enako kot v C-ju: `pogoj ? vrednost_če_res : vrednost_če_ne`) – pri branju izberemo podatek iz prvega ali drugega BRAM-a, glede na to, kateri kanal je bil naslovljen.

Torej je to preprost "naslovni razpletalec" (angl. *address decoder*): glede na en bit naslova se odloči, v kateri od dveh fizičnih pomnilnikov gre pisanje/branje.

Od pomnilnika do DAC izhoda

Ko je vzorec funkcije enkrat shranjen v enem od teh dveh BRAM-ov, poskrbi blok `generate_top`, da se preko **decimacije generatorja** (podobna ideja kot decimacija pri zajemu – nadzorujemo, kako hitro se "sprehajamo" po vzorcih shranjene funkcije)

prava vrednost prebere iz pomnilnika in posreduje naprej na **DAC** (Digital-to-Analog Converter), ki digitalno število spremeni nazaj v analogno napetost na fizičnem izhodu ploščice. Tako lahko FPGA "predvaja" poljubno, vnaprej definirano obliko signala v zanki (npr. za krmiljenje ogledala po nekem vzorcu gibanja).

1.4 Komunikacija zajetega signala

To je – kot pravi tudi izvirna dokumentacija – najbolj zapleten del sistema, saj gre za prenos **velikih količin podatkov** iz FPGA logike v RAM pomnilnik procesorja, in sicer učinkovito ter brez izgub podatkov.

Zakaj GP port ni dovolj hiter?

GP (General Purpose) AXI vodilo, ki smo ga spoznali v poglavju 1.1, je zasnovano za enostavne, občasne ukaze ("nastavi register", "preberi register") – ne pa za neprekinjen, hiter prenos tisočev ali milijonov vzorcev na sekundo. Za to imamo na Zynq procesorju posebno, veliko hitrejše vodilo: **HP (High Performance) AXI port** (`S_AXI_HP0` na blokovni shemi), ki omogoča direkten, hiter dostop do glavnega RAM pomnilnika (DDR) – podobno kot DMA (*Direct Memory Access*) na navadnih računalnikih, kjer periferna naprava lahko piše direktno v RAM brez posredovanja procesorja pri vsakem posameznem podatku.

Zgradba bloka AXI_HP_top

Blok `Acquire_HP_top` (na blokovni shemi imenovan `AXI_HP_top`) je sestavljen iz dveh podblokov:

- **`axi_s2mm`** (*Stream to Memory-Mapped*) – to je Xilinxov standardni gradnik, ki znotraj sebe implementira ves zapleteni AXI protokol, potreben za pisanje podatkovnega toka (streama) v pomnilnik, mapiran na naslove (torej v navaden RAM). Ta gradnik skrbi za vse podrobnosti AXI standarda (burst prenosi, usklajevanje `valid`/`ready` signalov, upravljanje z naslovi), tako da se nam kot uporabnikom tega ni treba ukvarjati s temi nizko-nivojskimi podrobnostmi.
- **`axi_control`** – naša lastna logika, ki poskrbi za dve stvari: (1) shranjevanje zajetih vzorcev v vmesni **FIFO** pomnilnik in (2) odločanje, kdaj je čas, da se sproži prenos teh podatkov preko `axi_s2mm` v RAM.

Kaj je FIFO in zakaj ga potrebujemo?

FIFO (*First In, First Out*) je vrsta pomnilnika, ki deluje kot vrsta v trgovini: podatki, ki pridejo prvi, grejo tudi prvi naprej. FIFO je tukaj ključen kot **blažilnik (buffer)** med dvema deloma sistema, ki delujeta z različno "ritmiko":

- Zajemanje vzorcev (integrator + pozicije) se dogaja **neprestano in redno**, z ritmom, ki ga določa `dec_cnt` (glej poglavje 1.2).
- Prenos v RAM preko AXI HP porta pa se dogaja **v paketih** (burst-ih) in ni vedno takoj na voljo (vodilo je lahko trenutno zaposleno z drugim prenosom, ali pa čakamo, da se nabere dovolj podatkov za učinkovit prenos).

FIFO torej "lovi" vzorce, ki pritečejo neprestano, in jih "spusti naprej" v RAM, ko je za to primeren trenutek – s tem preprečimo izgubo podatkov, tudi če prenos v RAM občasno ni takoj na voljo.

Sestavljanje FIFO zapisa (Slika 8)

```
assign data_in[15:0]  = osc1_data;
assign data_in[31:16] = osc2_data;
assign data_in[63:32] = cnt_data;
```

Tukaj se trije ločeni signali (dva 16-bitna pozicijska signala hitrega ogledala `osc1_data` in `osc2_data`, ter 32-bitni integrirani signal `cnt_data`) združijo v **en sam 64-bitni zapis** (`data_in`). To imenujemo *bit-konkatenacija* (združevanje) – posamezni signali se preprosto "zložijo" en ob drugega na določena mesta znotraj širšega vodila. Ko se ta 64-bitni zapis kasneje prebere na drugi strani (v C programu na procesorju), ga bo treba spet "razstaviti" na posamezne komponente – to natančno vidimo v C programu, poglavje 3.2, kjer imamo strukturo z ločenimi poljji za `adc_data_ch1`, `adc_data_ch2` in integrirani signal.

Zapis v FIFO (Slika 9)

```
fifo_generator_64_64 FIFO(
    .srst        (fifo_rst),
    .clk         (clk),
    .din         (data_in),
    .wr_en       (trigger),
    .dout        (data_out),
    .rd_en       (transfer),
    .data_count  (fifo_rd_cnt),
    .full        (full)
);
```

`fifo_generator_64_64` je Xilinxov vgrajen FIFO gradnik (ime pove, da je 64 bitov širok in globok 64 mest – ali podobno, odvisno od konfiguracije). Vhodi/izhodi:

- `din` / `wr_en` – podatek za vpis in signal "vpiši zdaj" (aktiviran s signalom proženja `trigger`, ki je isti signal kot `trig` iz poglavja 1.2 – vsak nov vzorec integratorja in pozicij se takoj vpiše v FIFO).
- `dout` / `rd_en` – podatek za branje in signal "preberi zdaj" (aktiviran s signalom `transfer`, ki pomeni "zdaj prenašamo podatke naprej v RAM").
- `data_count` (`fifo_rd_cnt`) – pove, koliko vzorcev je trenutno "v vrsti" v FIFO-ju, še ne prenesenih naprej.
- `full` – opozorilni signal, ki pove, da je FIFO povsem poln (v dobro načrtovanem sistemu se to ne bi smelo zgoditi, ker bi to pomenilo izgubo podatkov).

Zastavice za iniciacijo prenosa (Slika 10)

```
assign data_valid       = buffer_n > fifo_transfer_rd_cnt;
assign initiate_transfer = (fifo_rd_cnt > fifo_min_thresh) && (~busy) && (~request_inte
assign request_interrupt = ctrl_reg[REQUEST_INTR_ADDR] && data_valid;
```

To so tri ključne logične enačbe, ki uravnavajo celoten cikel prenosa:

- **`data_valid`** je aktiven, kadar še nismo prenesli vseh podatkov, ki jih želimo (primerjamo ciljno število `buffer_n` s trenutnim štetjem že prenesenih vzorcev `fifo_transfer_rd_cnt`) – torej "še je kaj za prenesti".
- **`initiate_transfer`** ("sproži prenos") postane aktiven, kadar je hkrati izpolnjenih več pogojev:
  - v FIFO-ju je nabranih **dovolj** vzorcev (`fifo_rd_cnt > fifo_min_thresh` – presežen minimalni prag, se pravi ne prenašamo prezgodaj/premalo učinkovito),
  - sistem ravno **ni** zaposlen z drugim prenosom (`~busy`),
  - in **ni** že čakajoča prošnja za prekinitev (`~request_interrupt`) – torej počakamo, da procesor pobere prejšnji paket podatkov, preden začnemo pripravljati novega.
- **`request_interrupt`** postane aktiven, kadar je v kontrolnem registru (`ctrl_reg`) postavljen bit, ki pove "želimo prekinitev, ko bo naslednjič na voljo dovolj podatkov" **in** je `data_valid` (torej dejansko obstajajo podatki za prenesti). To pove FPGA-ju: "ko boš pripravljen s prenosom, obvesti procesor preko interrupta."

Interrupt – kako FPGA "pokliče" procesor

Interrupt (prekinitev) je mehanizem, s katerim strojna oprema procesorju sporoči "zgodilo se je nekaj, kar potrebuje tvojo pozornost – takoj, ne šele čez nekaj časa". Brez interruptov bi moral procesor neprestano "spraševati" (angl. *polling*) FPGA, ali je nov paket podatkov že pripravljen, kar bi po nepotrebnem porabljalo procesorski čas. Z interrupti lahko procesor namesto tega "spi" (čaka), dokler ga FPGA aktivno ne "prebudi" s signalom `intr`.

Cel cikel torej poteka takole:

1. Vzorci se neprestano nabirajo v FIFO.
2. Ko jih je dovolj (presežen prag) in sistem ni zaposlen, se sproži prenos preko `axi_s2mm` v RAM (`initiate_transfer`).
3. Ko je prenos zaključen, se postavi zastavica `intr` na 1, kar fizično sproži signal na vodilu `IRQ_F2P` (Interrupt From FPGA to Processor) procesorja.
4. Na strani Linuxa program "čaka" na ta interrupt (glej poglavje 3.3), ga zazna, prebere podatke iz RAM-a in ponastavi zastavico `request_interrupt` na 1, s čimer FPGA-ju sporoči "pripravljen sem na naslednji paket".
5. Cikel se ponovi od 1. točke.

Ta pristop (FIFO + prag + interrupt) je zelo pogost vzorec pri načrtovanju hitrih vgrajenih sistemov, saj omogoča učinkovit, neprekinjen prenos podatkov brez izgub in brez po nepotrebnem zasedenega procesorja.

2 DeviceTree

To poglavje razloži, kako Linux “izve”, da v sistemu obstajajo naše FPGA naprave, in kako lahko do njih dostopamo iz navadnega C programa, ki teče v uporabniškem prostoru (ne v jedru Linuxa).

Zakaj Linux potrebuje “DeviceTree”?

Na navadnem osebnem računalniku operacijski sistem večino naprav odkrije samodejno (npr. preko PCI ali USB standardov, kjer se naprave same “predstavijo”). Na vgrajenih sistemih (kot je Zynq na RedPitayi) pa so mnoge naprave (predvsem naša lastna FPGA logika) povsem **nestandardne** in jih Linux ne more sam odkriti – nima od kod izvedeti, da na naslovu `0x40000000` obstaja neka naša naprava za zajem signalov.

Za to se uporablja **DeviceTree** – to je preprosta, strukturirana datoteka (podobna XML/JSON po duhu, čeprav ima svojo lastno sintaktiko), v kateri **mi sami** povemo Linuxu: “na tem naslovu v pomnilniškem prostoru obstaja naprava, ki komunicira takole, in uporablja ta interrupt.”

Kako se naslovi ujemajo med Vivadom in DeviceTree (Slika 11)

Ko v Vivadu (orodje za načrtovanje FPGA) sestavljamo sistem iz blokov (kot smo videli v poglavju 1, blokovna shema `system.pdf`), ima orodje zavihek **Address Editor** (na sliki imenovan “Address manager”), kjer Vivado samodejno (ali z našo pomočjo) dodeli **naslovni prostor** vsakemu bloku, ki komunicira preko AXI vodila. To je nujno, ker mora vsak AXI “master” (npr. procesor) vedeti, na katerem naslovu se nahaja vsak posamezni “slave” (naša FPGA logika), da lahko z njim komunicira.

Recimo, da Vivado dodeli našemu `reg_ctrl_top` bloku (dostopnem preko GP porta) naslovni prostor, ki se začne na `0x40000000`, in `AXI_HP_top` bloku (dostopnem preko HP porta) naslovni prostor na `0x19000000`. Te iste vrednosti naslovov **morajo** biti kasneje navedene tudi v DeviceTree datoteki – če se ne ujemajo, Linux bo “gledal” na napačno mesto v pomnilniškem prostoru in komunikacija ne bo delovala.

DeviceTree overlay (Slika 12)

DeviceTree **overlay** je posebna vrsta DeviceTree datoteke, ki se lahko naloži **med delovanjem** sistema (ni potrebno ponovno zagnati celotnega Linuxa) – “prekrije” (overlay) obstoječi opis strojne opreme z dodatnimi napravami. To je zelo priročno pri razvoju FPGA sistemov, kjer pogosto spreminjamo strojno konfiguracijo (novo FPGA sliko naložimo) in ne želimo vsakič ponovno zaganjati celotnega operacijskega sistema.

Oglejmo si strukturo iz Slike 12, razdeljeno na dele:

```
/dts-v1/;
/plugin/;
/ {
    fragment@uio {
```

```
        target = <&amba_pl>;
        __overlay__ {

            rp_acquire: rp_acquire@50000000 {
                #address-cells = <1>;
                #size-cells = <1>;
                compatible = "generic-uio";
                reg = <0x40000000 0x1000>,<0x19000000 0x80000>;
                reg-names = "regset", "buffer";
                interrupt-parent = <&intc>;
                interrupts = <0 30 1>;
            };

            rp_generate: rp_generate@50100000 {
                ...
                reg = <0x40001000 0x1000>,<0x40002000 0x10000>;
                reg-names = "regset", "waveform";
                interrupt-parent = <&intc>;
                interrupts = <0 31 1>;
            };
        };
    };
};
```

Razlaga posameznih delov:

- `/dts-v1/;` – pove, da gre za DeviceTree datoteko verzije 1 (standard format).
- `/plugin/;` – oznaka, da gre za **overlay** (dodatek k obstoječemu drevesu naprav), ne za celoten opis od začetka.
- `fragment@uio { target = <&amba_pl>; ... }` – "ta fragment naj se pripne na del drevesa, ki se imenuje `amba_pl`" (`amba_pl` je standardno vozlišče, ki predstavlja programirljivo logiko – *Programmable Logic* – torej naš FPGA del sistema, v nasprotju z `amba_apu`, ki predstavlja fiksni ARM procesorski del).
- `rp_acquire: rp_acquire@50000000 { ... }` – to definira **eno napravo**, ki jo poimenujemo `rp_acquire`. Številka `@50000000` v imenu je zgolj poljubna oznaka (naslovna "nalepka" v drevesu naprav), ne dejanski fizični naslov (ta je definiran spodaj, v vrstici `reg`).
- `compatible = "generic-uio";` – **zelo pomembna vrstica**. Pove Linuxu, kateri gonilnik naj uporabi za to napravo. `"generic-uio"` pomeni: "uporabi splošni UIO gonilnik" (UIO = *Userspace I/O* – gonilnik, ki omogoča, da s napravo upravljamo direktno iz uporabniškega programa, brez potrebe po pisanju lastnega gonilnika jedra – glej poglavje 3.1 za več o tem).
- `reg = <0x40000000 0x1000>,<0x19000000 0x80000>;` – to je **srce** te definicije: seznam (naslov, velikost) parov. Prvi par (`0x40000000`, velikost `0x1000` = 4096 bajtov) je fizični naslov in velikost območja pomnilnika za **register** (kontrolni parametri – GP vodilo, glej poglavje 1.1). Drugi par (`0x19000000`, velikost `0x80000` = 512 KB) je naslov in velikost območja za **buffer** (podatke, ki jih prenaša HP vodilo, glej poglavje 1.4). Ta številski naslovi se morajo natančno ujemati s tistimi,

ki jih je Vivado dodelil v Address Editorju (Slika 11)!
- `reg-names = "regset", "buffer";` – ker imamo dve območji (dva para v `reg`), jim tu dodamo human-readable imena, da jih lahko v C kodi kasneje enostavno razlikujemo po imenu, ne le po vrstnem redu.
- `interrupt-parent = <&intc>;` – pove, kateri "nadzornik prekinitev" (*interrupt controller*, `intc`) je odgovoren za posredovanje interruptov te naprave.
- `interrupts = <0 30 1>;` – definira **kateri konkretni interrupt** (v tem primeru številka 30 v procesorjevem naboru IRQ-jev, ki ustreza signalu `IRQ_F2P[15:0]` iz FPGA – glej poglavje 1.4) je povezan s to napravo, in kakšnega tipa je (zadnja številka, `1`, določa način sprožitve, npr. na naraščajoči rob).

Podoben blok se ponovi za `rp_generate` (naprava za generiranje signala, poglavje 1.3), le z drugimi naslovi (`0x40001000` za registre in `0x40002000` za pomnilnik valovnih oblik) in drugo interrupt številko (31).

Kompilacija in nalaganje overlay-a

Dokumentacija omenja dve skripti:

- **`compile_dt.sh`** – DeviceTree datoteke se pišejo v berljivi (textovni) obliki `.dts`, vendar jih Linux jedro ne razume direktno – potrebuje binarno obliko `.dtbo` (Device Tree Blob Overlay). Ta skripta požene orodje `dtc` (Device Tree Compiler), ki `.dts` datoteko prevede v binarno `.dtbo` datoteko.
- **`apply_overlay.sh`** – ta skripta binarno `.dtbo` datoteko dejansko **naloži** v delujoči Linux sistem (preko posebnega mehanizma v `/sys/kernel/config/device-tree/overlays/` ali podobno), s čimer se nova naprava takoj "pojavi" v sistemu, brez potrebe po ponovnem zagonu.

Ko je overlay enkrat naložen, se v mapi `/dev/` (in `/sys/class/uio/`) pojavijo nove naprave (npr. `/dev/uio0`, `/dev/uio1`), s katerimi lahko nato upravljamo iz C programa – to je natanko tema naslednjega poglavja.

3 C program

Ko je strojna oprema (FPGA logika, poglavje 1) sprogramirana in jo Linux "vidi" (poglavje 2), nam preostane še napisati **uporabniški program** (v jeziku C), ki bo z njo dejansko upravljal: nastavljal parametre, zajemal podatke in jih pošiljal naprej preko omrežja.

Kaj je UIO in zakaj ga uporabljamo?

UIO (Userspace I/O) je splošen mehanizem v Linux jedru, ki omogoča **varno** upravljanje s strojno opremo **direktno iz navadnega uporabniškega programa** (ne iz jedra). Brez UIO bi morali za vsako našo posebno FPGA napravo napisati poseben gonilnik jedra (*kernel driver*) – kar je zapleteno, tvegano (napaka v gonilniku jedra lahko sesuje cel sistem) in zahteva veliko več znanja o notranjosti Linux jedra.

UIO namesto tega naredi nekaj genialno preprostega: jedru pove le "ta naprava obstaja na tem naslovu in ima ta interrupt", nato pa preprosto **razkrije** ta pomnilniški prostor uporabniškemu programu preko standardnih datotečnih operacij (`open()`, `mmap()`, `read()`), podobno kot bi odprli navadno datoteko. Uporabniški program lahko nato ta pomnilniški prostor preslika (*mmap – memory map*) direktno v svoj naslovni prostor in do njega dostopa enako enostavno, kot bi dostopal do navadne spremenljivke ali polja (array) v C-ju – čeprav v resnici "pod pokrovom" vsak dostop do tega pomnilnika neposredno vpliva na FPGA logiko!

3.1 Identifikacija UIO naprav

Ker je lahko na sistemu prisotnih več UIO naprav (v našem primeru vsaj dve: ena za zajem `rp_acquire` in ena za generiranje `rp_generate` – glej poglavje 2), moramo v C programu najprej poiskati **prave** naprave med vsemi, ki so na voljo.

Kako Linux poimenuje UIO naprave

Ko je overlay naložen, se v `/dev/` pojavijo naprave z imeni `uio0`, `uio1`, `uio2`, itd. – ta imena so dodeljena **po vrstnem redu odkritja**, ne po imenu, ki smo ga uporabili v DeviceTree (`rp_acquire`, `rp_generate`). To pomeni, da se lahko `rp_acquire` enkrat pojavi kot `/dev/uio0`, po ponovnem zagonu pa morda kot `/dev/uio1` – vrstni red ni zagotovljen!

Da se izognemo temu problemu, Linux za vsako UIO napravo v `/sys/class/uio/uioX/name` shrani tudi **ime**, ki smo ga definirali v DeviceTree (v našem primeru `rp_acquire` ali `rp_generate`). Zato program v datoteki `uio_parser.h` naredi naslednje:

1. Preišče vse mape v `/sys/class/uio/` (torej `uio0`, `uio1`, ...).
2. Za vsako od njih prebere datoteko `name`, da ugotovi, kako se ta konkretna naprava dejansko imenuje (npr. `rp_acquire`).
3. Sestavi seznam vseh najdenih naprav skupaj z njihovimi imeni in podrobnostmi.
4. Ta seznam nato program preišče in poišče vnos, katerega ime se ujema s tistim, kar išče (npr. `"rp_acquire"`), in tako pridobi pravilno pot (npr. `/dev/uio0`), ne glede na to, katera številka je bila dejansko dodeljena.

Ta pristop (iskanje po imenu namesto po fiksni številki) je pomemben princip robustnega programiranja vgrajenih sistemov: sistem naj se ne zanaša na vrstni red, ki ni zagotovljen, ampak na eksplicitno, stabilno identifikacijo (ime naprave).

Več "map" (`maps`) znotraj ene UIO naprave

Spomnimo se iz poglavja 2, da ima npr. `rp_acquire` **dva** ločena naslovna prostora (`reg` z imenoma `"regset"` in `"buffer"`). UIO to izpostavi kot dve ločeni "mapi" (angl. *maps*) znotraj iste naprave – v Linuxu vidne kot `/sys/class/uio/uioX/maps/map0/` in `.../map1/`. To je razlog, zakaj v `uio_parser.h` obstaja struktura, ki za vsako UIO napravo hrani **seznam** (verižni seznam, `uio.maps->next`) njenih map, ne le ene same – vsaka naprava ima lahko poljubno število ločenih pomnilniških območij.

3.2 Mapiranje registrov in bufferja

Ko smo v prejšnjem korak najprej pravo UIO napravo, jo moramo dejansko **preslikati (mapirati)** v naslovni prostor našega C programa, da lahko do njenih registrov dostopamo kot do navadnih spremenljivk.

Funkcija za dodajanje generatorja (Slika 13)

```
Gen *create_gen(Uio uio) {
    char *path = malloc(sizeof(char[100]));
    snprintf(path, 100, "%s%s", "/dev/", uio.name);
    printf("%s\n", path);

    int fd = open(path, O_RDWR);

    Reg_map_gen *reg = memory_map(fd, uio.maps->uio_map.size, 0);
    Waveform_container *waveform = memory_map(fd, uio.maps->next->uio_map.size, 1);

    Gen *gen = malloc(sizeof(Gen));

    gen -> reg = reg;
    gen -> waveform_container = waveform;
    gen -> fd = fd;

    return gen;
}
```

Razlaga po korakih:

1. **`snprintf(path, ...)`** sestavi pravo pot do datoteke naprave, npr. `/dev/uio1` (glede na ime, ki smo ga našli v prejšnjem koraku).
2. **`open(path, O_RDWR)`** odpre to napravo, kot bi odprli navadno datoteko – `O_RDWR` pomeni "za branje in pisanje". To vrne t. i. **file descriptor** (`fd`) – številko, s katero se nato sklicujemo na to odprto napravo pri vseh nadaljnjih operacijah.

3. **`memory_map(fd, ..., 0)`** in **`memory_map(fd, ..., 1)`** – to je funkcija (definirana drugje v kodi, verjetno kot ovojnica okoli sistemskega klica `mmap()`), ki **preslika** določeno "mapo" (glej prejšnje podpoglavje – map0 ali map1, zato zadnji parameter 0 oz. 1) v naslovni prostor našega programa. Rezultat te preslikave je **kazalec** (pointer), ki ga lahko uporabljamo, kot bi kazal na navadno strukturo v pomnilniku – čeprav v resnici kaže direktno na registre v FPGA!
4. Prva preslikava (`reg`, tip `Reg_map_gen*`) je preslikava **prve** mape ("regset" – kontrolni parametri, glej poglavje 1.1 in DeviceTree definicijo `reg-names = "regset", "waveform";`).
5. Druga preslikava (`waveform`, tip `Waveform_container*`) je preslikava **druge** mape ("waveform" – dejanski vzorci funkcije, glej poglavje 1.3).
6. Na koncu se vse to (oba kazalca + `fd`) shrani v novo strukturo `Gen`, ki jo funkcija vrne – to je "ročaj" (handle), preko katerega bo preostanek programa upravljal s to napravo.

Zakaj deluje `mmap()` tukaj tako "magično"?

Ključna ideja `mmap()` sistemskega klica na Linuxu je, da lahko **datoteko** (ali v tem primeru posebno UIO "datoteko", ki v resnici predstavlja pomnilniško območje strojne opreme) preslikamo neposredno v naslovni prostor procesa, tako da se **vsak dostop do te preslikane pomnilniške regije v resnici prevede v neposreden dostop do fizičnega pomnilnika/registra**. To pomeni, da ko v C-ju napišemo npr. `gen->reg->start_gen = 1;`, se ta zapis **fizično** zgodi na točno tistem naslovu v FPGA-ju, ki ustreza registru `START_GEN_ADDR` (glej Sliko 3 iz poglavja 1) – brez kakršnihkoli vmesnih "prevodov" ali funkcijskih klicev. To je izjemno elegantno in hitro, saj se C struktura obnaša popolnoma enako kot navadna spremenljivka, čeprav v ozadju neposredno krmili strojno opremo.

Definicija struktur (Slika 14)

```
typedef volatile struct {
    uint32_t start_gen;
    uint32_t dec_factor_1;
    uint32_t dec_factor_2;
    uint32_t waveform_len_1;
    uint32_t waveform_len_2;
} Reg_map_gen;

typedef volatile struct {
    int16_t waveform_1[MAX_WAVEFORM_LEN];
    int16_t waveform_2[MAX_WAVEFORM_LEN];
} Waveform_container;

typedef struct {
    Reg_map_gen *reg;
    Waveform_container *waveform_container;
    int fd;
```

```
} Gen;
```

Nekaj ključnih opažanj:

- Ključna beseda **`volatile`** je tukaj **absolutno nujna**. Pove prevajalniku (compiler): "ne optimiziraj dostopov do te spremenljivke, ne predvidevaj, da se njena vrednost ne spremeni sama od sebe med dvema branjema – vedno dejansko preberi/zapiši iz/v pomnilnik." Brez `volatile` bi lahko prevajalnik (ker "ne vidi", da se ta pomnilnik lahko spremeni od zunaj, s strani FPGA logike) po nepotrebnem "predpomnil" vrednost v procesorjev register in nikoli več dejansko prebral iz pomnilnika, kar bi popolnoma pokvarilo delovanje – program bi vedno "videl" staro, zamrznjeno vrednost.
- Vrstni red in tipi polj v strukturi `Reg_map_gen` **morajo natančno ustrezati** vrstnemu redu in širini (32-bitna `uint32_t`) parametrov, kot so definirani v Verilogu (Slika 3, naslovi `START_GEN_ADDR`, `DECIMATION_GEN_ADDR_1`, `DECIMATION_GEN_ADDR_2`, `WAVEFORM_LEN_1_ADDR`, `WAVEFORM_LEN_2_ADDR` – vidimo, da se vrstni red polj v strukturi ujema z vrstnim redom naslovov!). To je zato, ker C preprosto razporedi polja strukture eno za drugim v pomnilniku, brez vmesnih presledkov (če ni posebnih navodil za *padding*) – zato mora programer **ročno** poskrbeti, da se ta razporeditev natančno ujema s tem, kar pričakuje FPGA stran.
- `Waveform_container` vsebuje dve polji (array) tipa `int16_t` (16-bitno predznačeno število), kar se natančno ujema s tem, da so vzorci funkcije 16-bitni (glej `waveform_val_1[15:0]` na blokovni shemi) in da imamo dva ločena kanala (spomnimo se delitve na 16384 + 16384 vzorcev iz poglavja 1.3).
- Struktura `Gen` je "ovoj" (wrapper), ki združi oba kazalca (na registre in na vzorce) skupaj s `fd`, tako da lahko preostanek programa z eno samo spremenljivko (`Gen*`) dostopa do vsega, kar se tiče generatorja signala.

Popolnoma analogen sistem (torej svoja struktura, npr. `Reg_map_acq` in ustrezen "container" za zajete podatke) obstaja tudi za `acquire/oscilloscope` stran, definiran v `oscilloscope.h`, ki ga dokumentacija omenja, vendar ne prikaže eksplicitno – princip je pa povsem enak.

3.3 Prenos FIFO v RAM

To poglavje pokaže, kako se v C programu **ujame** interrupt, ki ga FPGA sproži, ko je nov paket podatkov pripravljen v RAM-u (glej poglavje 1.4 za FPGA stran te zgodbe).

Poslušanje za interrupt (Slika 15)

```
while (wait(acq) == 0) {
    len = acq->reg->buff_n*8;

    write(connfd, acq->buff, len);
    request_and_clear_interrupt(acq);
}
```

- **`wait(acq)`** je funkcija (definirana drugje v kodi, verjetno v `oscilloscope.h`), ki

znotraj sebe kliče sistemski klic `read()` na UIO datoteki naprave (`acq->fd`). To ni "aktivno spraševanje" (*polling*) – klic `read()` na UIO datoteki **blokira** (torej program "zaspi" in ne porablja procesorskega časa), dokler se ne zgodi dejanski interrupt s strani FPGA. Ko se interrupt zgodi, jedro Linuxa "prebudi" naš program in `read()` (oz. `wait()`) se vrne.
- `while (wait(acq) == 0)` – zanka, ki se ponavlja **v nedogled**, vsakič ko pride nov interrupt (pogoj `== 0` je verjetno preverjanje, da ni prišlo do napake – konvencija, kjer 0 pomeni "uspešno").
- **`len = acq->reg->buff_n * 8;`** – izračuna dolžino podatkov, ki jih je treba prenesti. Ker je vsak zapis v FIFO 64-biten (8 bajtov – spomnimo se Slike 8, kjer smo sestavili 64-bitni `data_in`), množimo število vzorcev (`buff_n`, prebran direktno iz registra preko preslikanega pomnilnika!) z 8, da dobimo skupno število **bajtov**.
- **`write(connfd, acq->buff, len);`** – to je standardni Linux sistemski klic za pisanje na **socket** (`connfd` je file descriptor omrežne povezave – glej poglavje 3.4), ki podatke iz bufferja (`acq->buff` – ta kazalec kaže direktno na preslikan RAM prostor, kamor je FPGA zapisal podatke preko HP porta!) pošlje po omrežju do povezanega klienta.
- **`request_and_clear_interrupt(acq);`** – po tem, ko smo podatke prenesli naprej, moramo FPGA obvestiti, da smo končali in da lahko pošlje naslednji paket (glej spodaj).

Zakaj je pomembno, da se to dogaja v neskončni zanki?

Ta zanka je "srce" celotnega delovanja programa na strani procesorja – ona neprestano čaka na nove podatke, jih prenaša naprej in signalizira FPGA-ju, da je pripravljena na naslednji paket. Ves preostali del programa (nastavljanje parametrov, inicializacija) se zgodi **pred** to zanko, sam prenos podatkov pa teče znotraj nje, dokler program ne konča (npr. ob prekinitvi s strani uporabnika).

Brisanje interrupta in zaprošanje za novega (Slika 16)

```
void request_and_clear_interrupt(Acq *acq) {
    set_reg(&(acq->reg->ctrl_reg), 0x00000003);
}
```

- **`set_reg(...)`** je pomožna funkcija (verjetno preprosto `*naslov = vrednost;` z dodatnimi varnostnimi preverjanji), ki zapiše novo vrednost v register preko preslikanega pomnilnika – natančno tako, kot smo razložili v poglavju 3.2 (dostop do `volatile` strukture, ki fizično vpliva na FPGA).
- Vrednost `0x00000003` v binarni obliki je `...011` – torej sta postavljena **dva najnižja bita**. Glede na FPGA kodo (poglavje 1.4, konstante `CTRL_REG_INTR_ADDR = 0` in `REQ_CTRL_REG_INTR_ADDR = 1` iz Slike 3) lahko sklepamo, da bit 0 nekako "potrdi/počisti" obstoječi interrupt (torej pove FPGA-ju "prejel sem prejšnji interrupt, lahko ga ponastaviš"), bit 1 pa ponovno **zaprosi** za naslednji interrupt (torej ustreza signalu `ctrl_reg[REQUEST_INTR_ADDR]` iz Slike 10, ki mora biti postavljen na 1, da se `request_interrupt` logika v FPGA-ju sploh lahko ponovno sproži).

To se natanko ujema s krogom, ki smo ga opisali v poglavju 1.4: FPGA čaka, da je `ctrl_reg[REQUEST_INTR_ADDR]` postavljen, preden lahko ponovno sproži `request_interrupt` in nov prenos podatkov – s tem, da procesor to zastavico **eksplicitno** ponovno postavi šele **po** tem, ko je varno prevzel prejšnji paket podatkov, se izognemo temu, da bi FPGA prehitro prepisal podatke, ki jih procesor še ni prebral (t. i. *race condition* – tekmovalno stanje, kjer bi dva dela sistema hkrati poskušala uporabljati isti vir podatkov, kar bi lahko privedlo do izgube ali poškodbe podatkov).

3.4 TCP komunikacija

Zadnji korak je, da zajete podatke dejansko **pošljemo na drug računalnik** (npr. osebni računalnik uporabnika), kjer se iz njih sestavi slika.

Kaj je TCP in zakaj se uporablja tukaj?

TCP (Transmission Control Protocol) je eden od osnovnih internetnih protokolov, ki poskrbi za **zanesljiv** prenos podatkov med dvema računalnikoma preko omrežja (npr. preko Etherneta, na katerega je RedPitaya povezana). "Zanesljiv" pomeni, da TCP samodejno poskrbi za:

- pravilni **vrstni red** prejetih podatkov (tudi če bi po omrežju "potovali" v drugačnem vrstnem redu),
- **ponovno pošiljanje** izgubljenih paketov (če se kakšen del podatkov med prenosom izgubi, TCP ga samodejno ponovno pošlje),
- osnovno **kontrolo pretoka** (angl. *flow control* – da hitrejši pošiljatelj ne "preplavi" počasnejšega prejemnika).

Iz vidika programerja je TCP povezava (`connfd` v kodi iz prejšnjega podpoglavja) videti skoraj enako kot navadna datoteka – enkrat ko je povezava vzpostavljena (kar se zgodi z dodatno kodo, ki v tej dokumentaciji ni podrobno prikazana, a bi tipično vključevala klice `socket()`, `bind()`, `listen()`, `accept()` na strani RedPitaye, torej "strežnika"), lahko podatke preprosto **pišemo** nanjo z `write()`, tako kot smo videli v Sliki 15.

Celoten potek podatkov, od FPGA do slike

Za popoln pregled sistema strnimo celotno pot enega vzorca podatkov, od začetka do konca:

1. **ADC** vzorči analogni signal iz detektorja in pozicijskega senzorja hitrega ogledala (poglavje 1.2).
2. **Integrator** sešteje prispevke signala iz detektorja znotraj enega decimacijskega okna (poglavje 1.2).
3. Ob signalu proženja (`trig`) se integrirani signal skupaj z obema pozicijskima signaloma združi v en 64-bitni zapis in shrani v **FIFO** (poglavje 1.4).
4. Ko je v FIFO-ju nabranih dovolj vzorcev, se preko **AXI HP** vodila podatki prenesejo v **RAM** procesorja (poglavje 1.4), FPGA nato sproži **interrupt**.
5. C program na strani Linuxa, ki čaka na ta interrupt (poglavje 3.3), se "prebudi", prebere podatke direktno iz preslikanega RAM prostora (poglavje 3.2) in jih

preko **TCP** poveza takoj posreduje naprej na osebni računalnik (poglavje 3.4).

6. Program nato zaprosi za naslednji interrupt in cikel se ponovi – neprestano, dokler zajem teče.
7. Na osebnem računalniku (izven te dokumentacije) se iz prejetega zaporedja integriranih vrednosti in pozicijskih signalov, glede na “zgornjo in spodnjo mejo pozicije” ter željeno **resolucijo**, sestavi slika – vsak vzorec integriranega signala postane ena “svetlobna” točka na sliki, na položaju, ki ga določata pozicijska signala (podobno kot pri elektronskem mikroskopu ali starih televizijskih zaslonih, kjer žarek “riše” sliko vrstico za vrstico, tu pa ogledalo “riše” sliko točka za točko).

S tem se zaokroži celotna slika sistema: od fizičnega analognega signala, preko FPGA logike, ki ga zajame in decimira, preko Linux gonilnikov (UIO) in DeviceTree opisa strojne opreme, do končnega C programa, ki podatke posreduje po omrežju za sestavljanje končne slike.

Povzetek najpomembnejših pojmov

Za lažje kasnejše iskanje strnimo najpomembnejše pojme, ki se pojavijo v tej dokumentaciji:

- **FPGA** – programirljiv čip, ki izvaja vzporedno strojno logiko, opisano v Verilogu.
- **Verilog `always @(posedge clk)`** – opis logike, ki se posodobi ob vsakem urinem taktu.
- **<= (neblokirajoča dodelitev)** – dodelitev vrednosti, ki se "izvrši" ob naslednjem urinem taktu, ne takoj.
- **AXI protokol (GP in HP)** – standardiziran način komunikacije med procesorjem in strojno opremo; GP za počasne ukaze/parametre, HP za hitre množične prenose podatkov.
- **Register** – poimenovano pomnilniško mesto v FPGA, dostopno preko naslova, preko katerega procesor nastavlja ali bere stanje strojne opreme.
- **Decimacija** – zmanjšanje hitrosti vzorčenja/generiranja s shranjevanjem/uporabo le vsakega n-tega vzorca.
- **Integrator** – blok, ki sešteva (integrira) vrednosti signala skozi čas, za zmanjšanje šuma.
- **BRAM (Block RAM)** – majhen, hiter pomnilniški blok znotraj FPGA čipa.
- **Dvo-vratni pomnilnik (dual-port RAM)** – pomnilnik z dvema neodvisnima dostopnima točkama, ki omogoča hkratno branje in pisanje brez medsebojnega oviranja.
- **FIFO** – vrsta pomnilnika "prvi noter, prvi ven", uporabljena kot blažilnik med različno hitrimi deli sistema.
- **Interrupt** – signal, s katerim strojna oprema obvesti procesor, da se je zgodilo nekaj, kar potrebuje takojšnjo pozornost.
- **DeviceTree (in overlay)** – opis strojne opreme, ki Linuxu pove, kje in kako do določene naprave dostopati; overlay omogoča dodajanje te informacije med delovanjem sistema.
- **UIO (Userspace I/O)** – Linux mehanizem, ki omogoča upravljanje s strojno opremo direktno iz uporabniškega programa, brez pisanja gonilnika jedra.
- **`mmap()`** – sistemski klic, ki pomnilniško območje strojne opreme preslika direktno v naslovni prostor uporabniškega programa.
- **`volatile`** (v C-ju) – ključna beseda, ki prevajalniku prepove optimizacijo dostopov do spremenljivke, ker se njena vrednost lahko spremeni "od zunaj" (s strani strojne opreme).
- **TCP** – zanesljiv omrežni protokol, uporabljen za prenos zajetih podatkov do osebnega računalnika, kjer se sestavi končna slika.